



Cenário Gold Miners

Programação de Agentes com JaCaMo para MAS

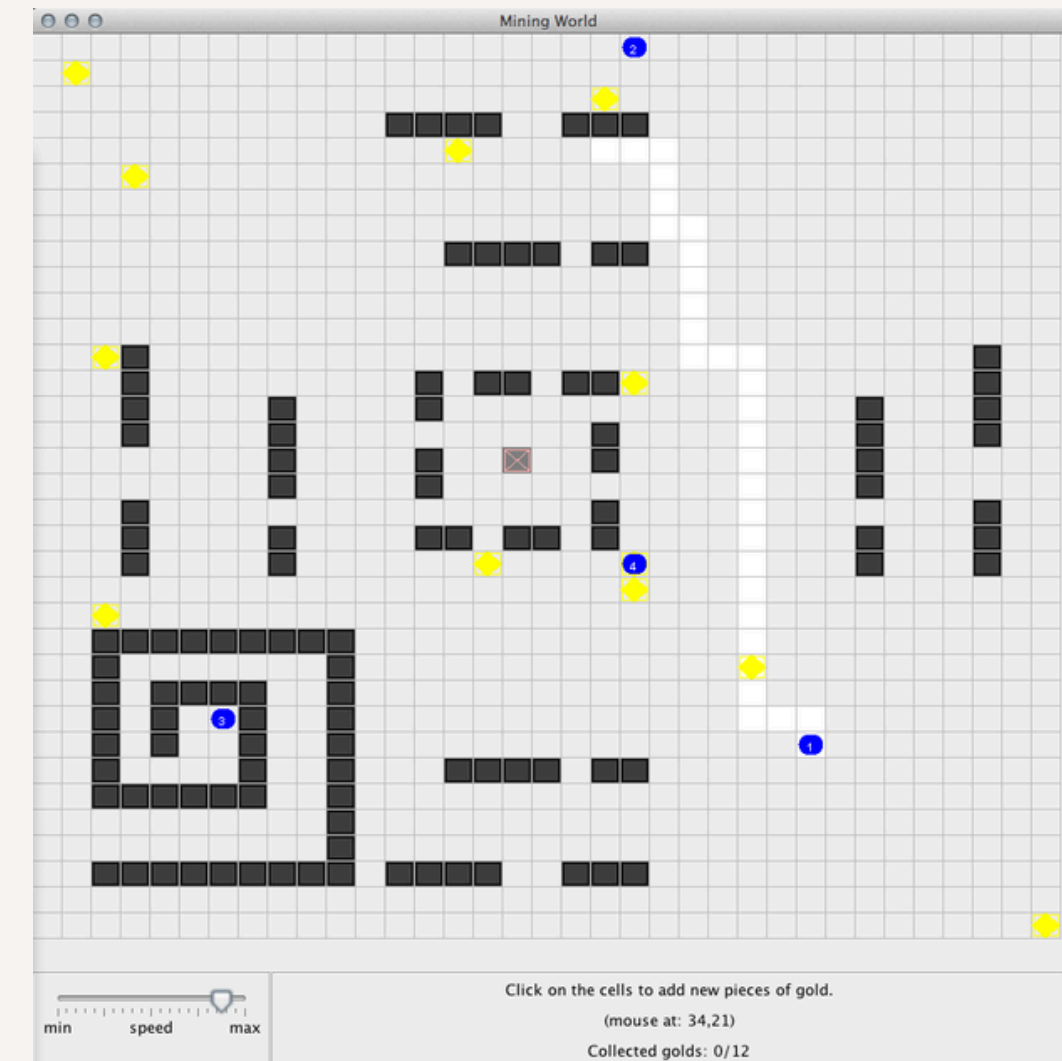
DANIEL VICTOR KREPSKY
JOÃO PEDRO DA SILVA CARDOSO
RAFAEL BERTOLINI PEREIRA SILVA

ESTRUTURA DA APRESENTAÇÃO

- 01 Introdução
- 02 Implementações da atividade do Gold Miners
- 03 Implementações extras propostas pelo grupo
- 04 Resultados
- 05 Conclusão

Introdução: Cenário e tarefas propostas pelo grupo

- Estudo do cenário:
 - agentes miner.asl;
 - leader.asl;
 - artefatos do mundo.
- Implementação de exercícios propostos no tutorial do gold miners;
- Criar extensões para o Gold Miners, a fim de deixar o cenário mais complexo;
- Testar e validar cada extensão individualmente, versionando a cada incremento;
- Criar estratégias diferentes na programação de decisão dos agentes para haver uma competição de três equipes;
- Documentar o que foi feito, tanto no relatório quanto em no repositório disponibilizado no github.



Fonte: <https://github.com/jacamo-lang/jacamo/blob/main/doc/tutorial/s/gold-miners/readme.adoc>



02

Implementações da atividade do gold miners

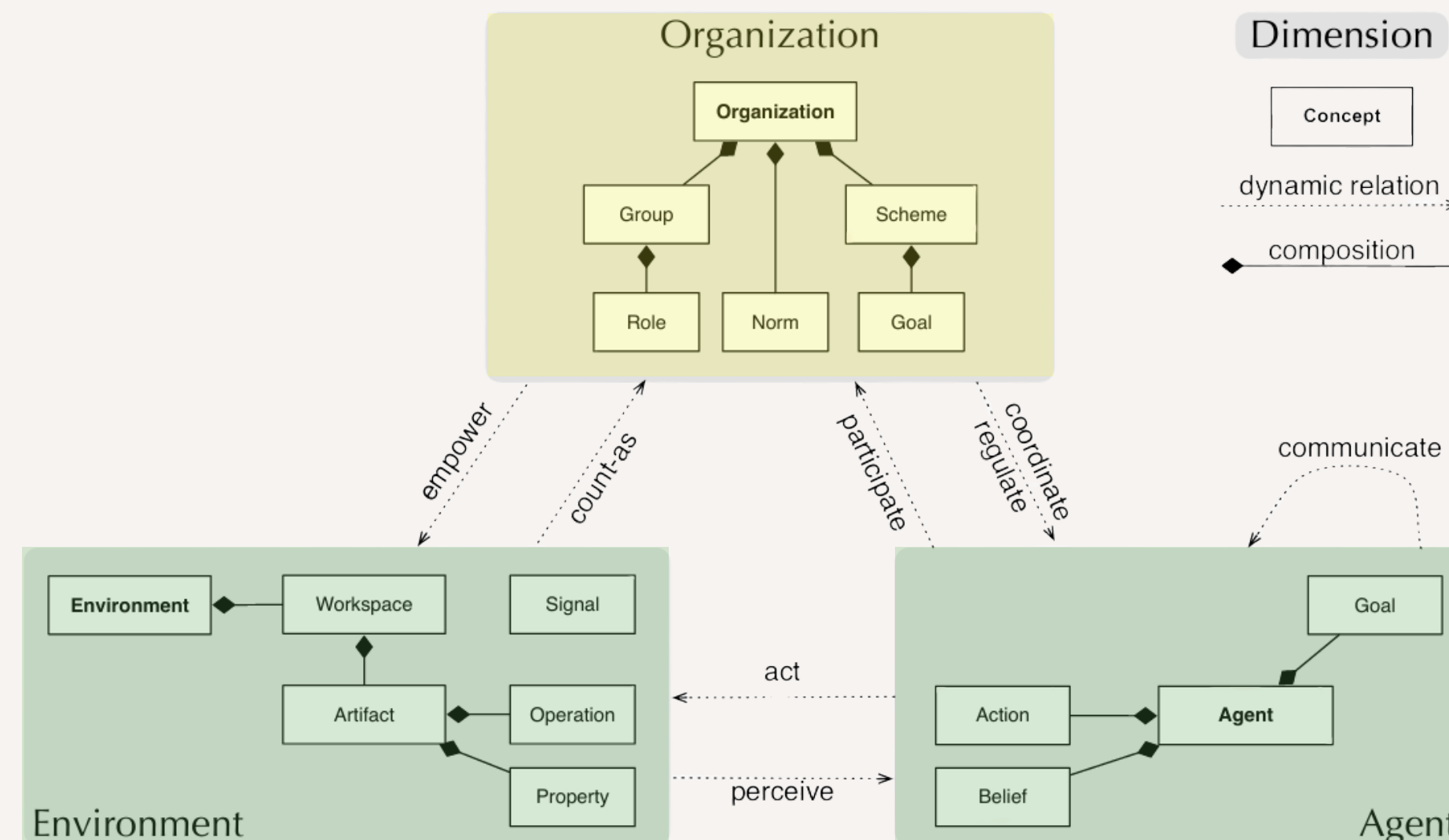
- Questões fundamentais para a compreensão do funcionamento dos agentes e do ambiente de mineração;
- Incluíram:
 - Adição da comunicação via log dos Miners;
 - Mudança de “Belief” → Baseado na distância do ouro;
 - Feedback dos Miners para o Leader sobre o ouro encontrado;
 - Histórico de Pontuação do ouro encontrado para cada Miner.
- Com esses exercícios, foi possível pensar em diversos possíveis incrementos que aumentariam a complexidade de busca e pontuação dos mineradores.



03

Implementações extras propostas pelo grupo

- As implementações foram feitas nas camadas de programação de agentes e de ambiente;
- Houve uma pequena implementação de organização, porém não possui complexidade hierárquica;
- As implementações iniciais foram mais simples, sendo incrementadas aos poucos para impleentações mais complexas (aquelas que alteravam de forma mais significativa, o ambiente de mineração e agentes).



Fonte: Boissier et al (2020)



03

Implementações extras propostas pelo grupo

Implementação de múltiplos minérios:

- Miner K – Múltiplos Minérios;

Funcionalidades distintas:

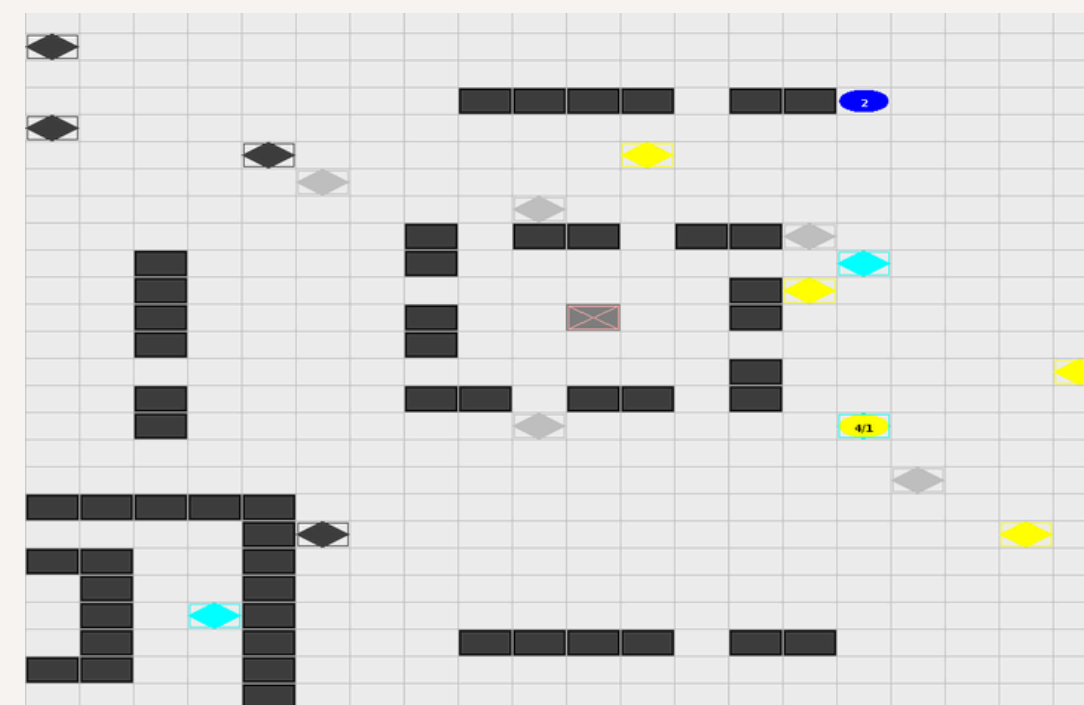
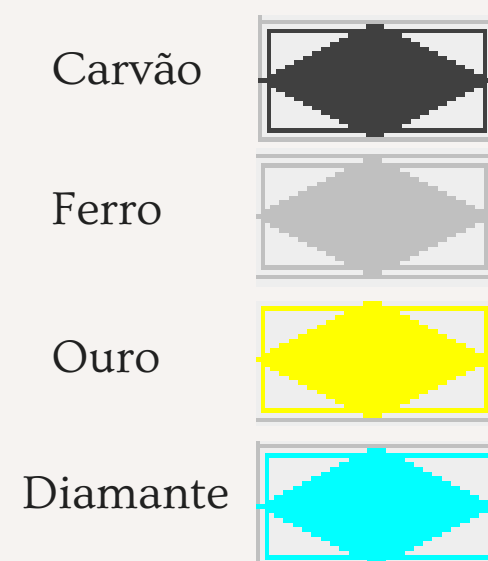
- Suporte à diferentes minérios e valores:
 - Carvão (1),
 - Ferro (2),
 - Ouro (3);
 - Diamante (4);

Arquivos atualizados:

- minerK.asl
- leaderK.asl
- WorldModelA.java
- MiningPlanetA.java
- WorldViewA.java

```
private void updateCellPercept(int x, int y) {  
    if (model == null || !model.inGrid(x,y)) return;  
  
    if (model.hasObject(WorldModelA.OBSTACLE, x, y)) {  
        defineObsProperty("cell", x, y, obstacle, 0);  
    } else if (model.hasObject(WorldModelA.COAL, x, y)) {  
        defineObsProperty("cell", x, y, coal, 1);  
    } else if (model.hasObject(WorldModelA.IRON, x, y)) {  
        defineObsProperty("cell", x, y, iron, 2);  
    } else if (model.hasObject(WorldModelA.GOLD, x, y)) {  
        defineObsProperty("cell", x, y, gold, 3);  
    } else if (model.hasObject(WorldModelA.DIAMOND, x, y)) {  
        defineObsProperty("cell", x, y, diamond, 4);  
    }  
}
```

Fonte: Elaborado pelos autores



Fonte: Elaborado pelos autores



03

Implementações extras propostas pelo grupo

Implementação do artefato cronômetro:

- Miner L – Cronômetro

Funcionalidades:

- Tempo de partida como input.
- Encerramento automático das ações ao finalizar o tempo.

Arquivos atualizados:

- minerL.asl
- leaderL.asl
- ClockArtifact.java

```
@INTERNAL_OPERATION
void countdown() {
    while (remaining > 0) {
        await_time(1000);
        remaining--;
        updateTimeLeft();
    }

    defineObsProperty("time_over");
}
```

Fonte: Elaborado pelos autores

```
[leader] Time left: 120 seconds
[leader] Time left: 119 seconds
[leader] Time left: 118 seconds
[leader] Time left: 117 seconds
[leader] Time left: 116 seconds
[leader] Time left: 115 seconds
[leader] Time left: 114 seconds
|
```

Fonte: Elaborado pelos autores



03

Implementações extras propostas pelo grupo

Implementação do artefato Rádio:

- Miner M – Rádio

Funcionalidades:

- Compartilhamento de localização dos minérios;
- Meio de comunicação entre agentes com restrição de sinal (blocos de distância).

Arquivos atualizados:

- minerM.asl
- leaderM.asl
- RadioArtifact.java

@OPERATION

```
void broadcastOre(int senderId, String oreType, int oreX, int oreY, int oreValue) {
    GridWorldModel model = getActiveModel();
    Location senderPos = model.getAgPos(senderId);

    for (int receiverId = 0; receiverId < NUM_MINERS; receiverId++) {
        if (receiverId == senderId) continue;

        Location receiverPos = model.getAgPos(receiverId);
        int distance = senderPos.distance(receiverPos);

        if (distance <= range && isSameTeam(senderId, receiverId)) {
            msgId++;
            getObsProperty("radio_ore_" + receiverId)
                .updateValues(senderId, new Atom(oreType), oreX, oreY, oreValue, msgId);
        }
    }
}

private boolean isSameTeam(int senderId, int receiverId) {
    String senderTeam = minerTeams.get(senderId);
    String receiverTeam = minerTeams.get(receiverId);
    return senderTeam != null && senderTeam.equals(receiverTeam);
}
```

Fonte: Elaborado pelos autores



03

Implementações extras propostas pelo grupo

Implementação do artefato Lanterna:

- Miner N – Lanterna

Funcionalidades:

- Ampliação do campo de visão.

Arquivos atualizados:

- minerN.asl
- LanternArtifact.java
- MiningPlanetA.java

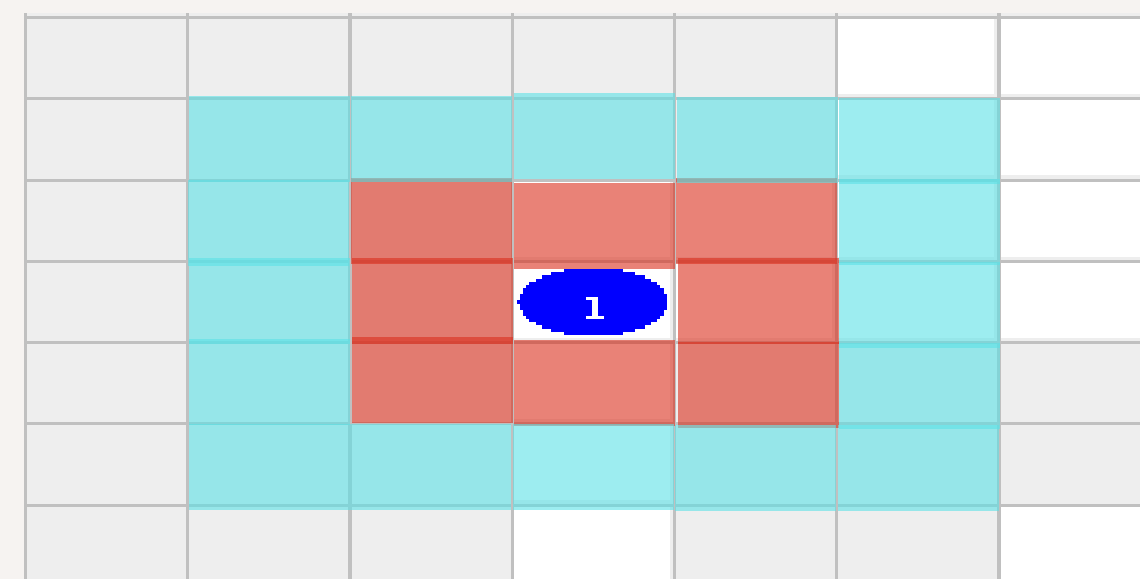
```
@OPERATION
void init(int radius) {
    visionRadius = radius;
    defineObsProperty("lantern_radius", visionRadius);
}

@OPERATION
void takeLantern(int minerId) {
    minersWithLantern.add(minerId);
}

public static boolean hasLantern(int minerId) {
    return minersWithLantern.contains(minerId);
}
```

Fonte: Elaborado pelos autores

 S/ lanterna
 +  C/ lanterna



Fonte: Elaborado pelos autores



03

Implementações extras propostas pelo grupo

Implementação do artefato Mochila:

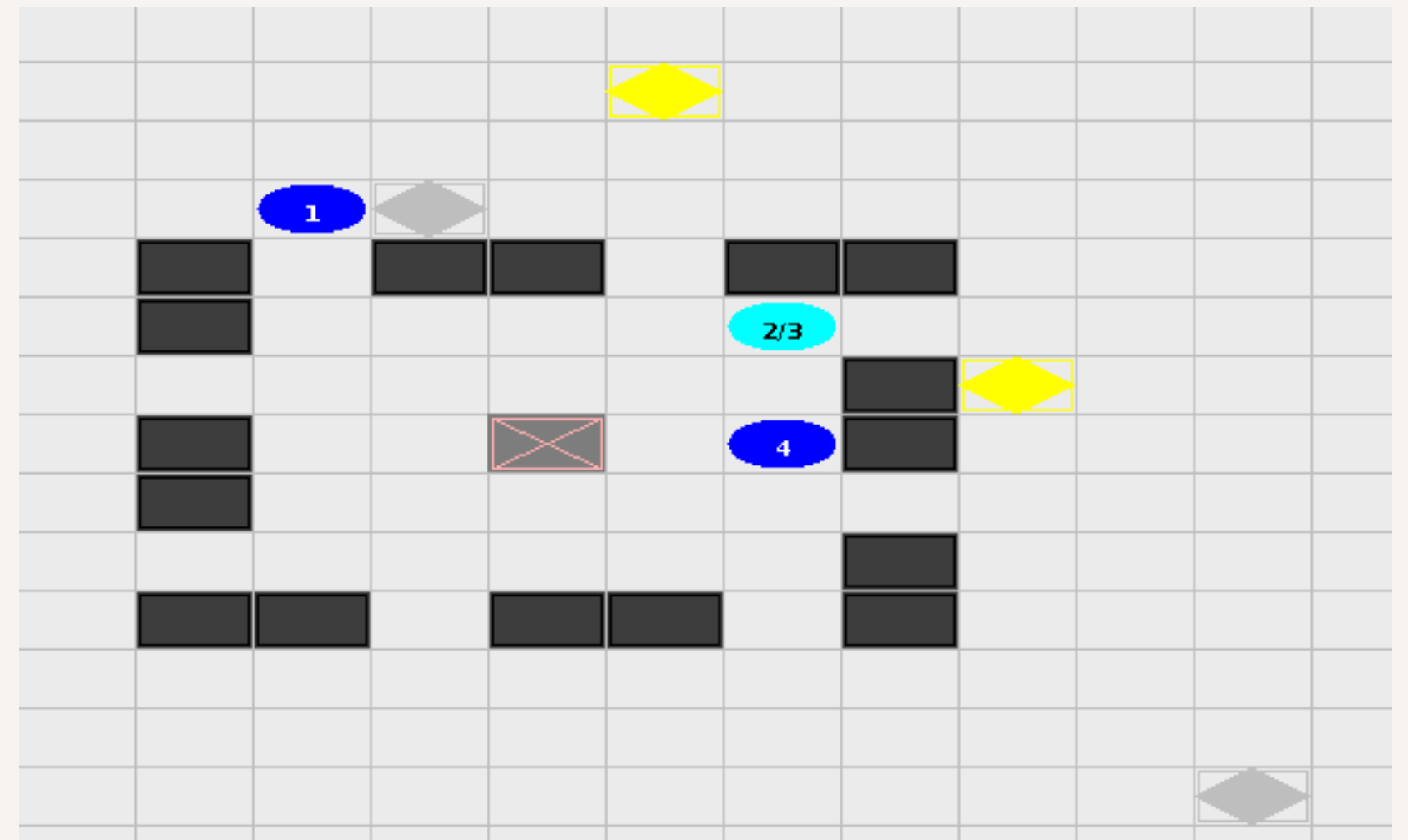
- Miner O – Mochila

Funcionalidades:

- Transporte de múltiplos minérios;
- Controle de capacidade.

Arquivos atualizados:

- minerO.asl
- BackpackArtifact.java
- WorldModelB.java
- MiningPlanetB.java
- WorldViewB.java



Fonte: Elaborado pelos autores



03

Implementações extras propostas pelo grupo

Implementação da organização simplificada com Moise:

- Miner P – Organização (Moise).

Funcionalidades:

- Times e papéis distintos (Explorer e Retriever);
- Comportamento baseado na função do agente.

Arquivos atualizados:

- minerP.asl
- org.xml

Structural Specification

Roles

- [retriever](#) extends [soc](#).
- [explorer](#) extends [soc](#).

Group *mining_team*

Possible roles: [retriever](#), [explorer](#).

Local links:

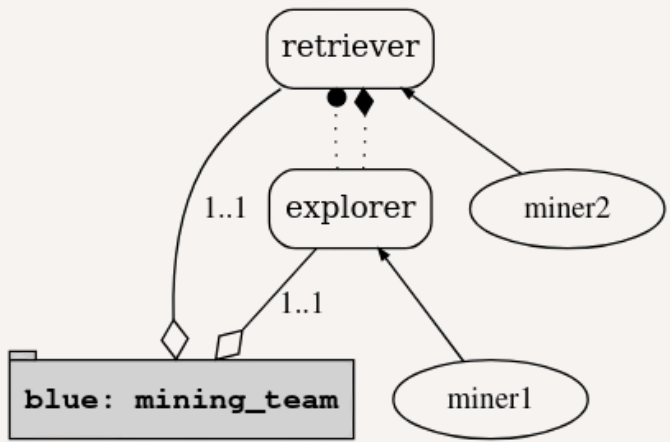
- [explorer](#) has a *communication* link to [retriever](#) and vice versa (intra-group, does not extend to subgroups)

blue (group)

created from specification [mining_team](#) (root group)

Formation:

ok



Players

- [miner1](#) plays [explorer](#)
- [miner2](#) plays [retriever](#)

Normative State

state	agent	maintenance condition	aim	deadline	done at	annotations
-------	-------	-----------------------	-----	----------	---------	-------------

[NPL program](#) / [debug page](#)



03

Implementações extras propostas pelo grupo

Competição entre três diferentes equipes:

Equipe **vermelha**

- Explorer se movimenta aleatoriamente;
- Retriever 1 se mantém próximo do explorer;
- Retriever 2 busca minérios aleatoriamente, e não escuta o rádio.

Equipe **azul**

- Explorer faz trajetos de busca total do mapa;
- Retriever 1 só volta quando a mochila estiver cheia;
- Retriever 2 coleta apenas 1 minério e entrega no depósito.

Equipe **verde**

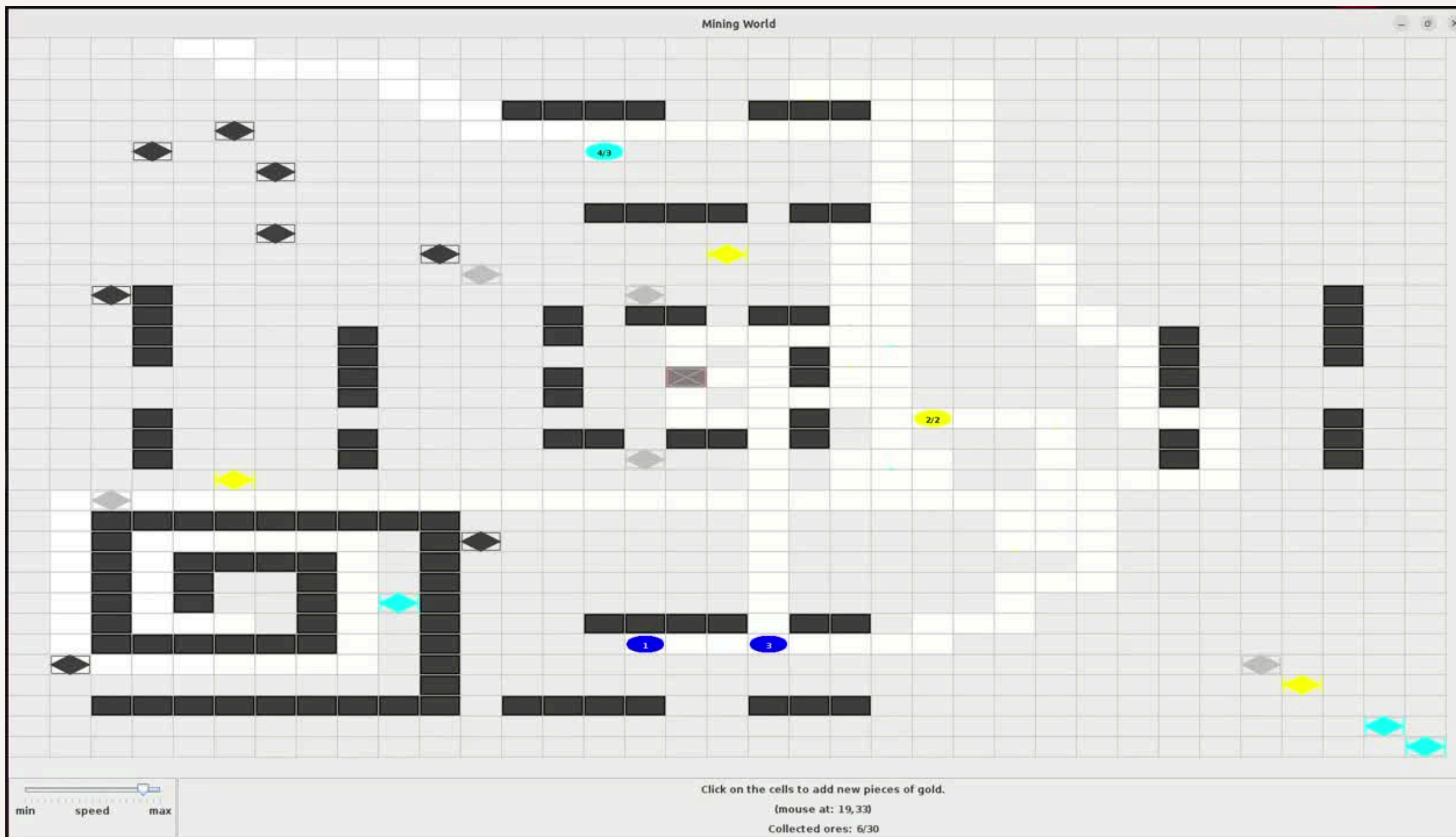
- Explorer comunica apenas minérios valiosos (ouro e diamante);
- Retriever 1 pega apenas diamante e ouro;
- Retriever 2 busca minérios aleatoriamente.

Os resultados obtidos entre as equipes serão discutidos no relatório final.



04

Resultados





05

Conclusão

- Implementação bem-sucedida das funcionalidades propostas e das extensões ao cenário original do Gold Miners;
- Evolução incremental do projeto, validando cada funcionalidade antes da integração da próxima;
- Ampliação da complexidade do ambiente por meio de múltiplos minérios, cronômetro, rádio, lanterna, mochila e organização com Moise;
- Desenvolvimento de agentes capazes de cooperar utilizando comunicação e divisão de papéis;
- Estrutura preparada para comparar diferentes estratégias entre equipes e apoiar futuros experimentos em Sistemas Multiagentes.

Referências

- Boissier, O., Bordini, R. H., Hübner, J. F., & Ricci, A. (2020). Multi-agent oriented programming: Programming multi-agent systems using JaCaMo. MIT Press.
- <https://jacamo-lang.github.io/>
- <https://github.com/jacamo-lang/jacamo/blob/main/doc/tutorials/gold-miners/readme.adoc>

Obrigado!

Caso hajam dúvidas:

Contato

krepsky.daniel@gmail.com;

joapedrosc2002@gmail.com;

rafael_bertolini_@hotmail.com